

University of Dhaka (DU)
Department of Computer Science & Engineering



DubsiBhai

Drainage Waterlogging Management for Dhaka

Survey Report

Course: CSE 4113 — Internet Programming Lab

Team: AI_TookMyJob

Team Members

Mehedi Hasan	Student ID: 22
Mahmudur Rahman Moin	Student ID: 30
Nafisha Akter	Student ID: 40
Biplob Paul	Student ID: 50

October 2026

Contents

1	Introduction	2
1.1	Background	2
1.2	Project Overview	2
1.3	Motivation	2
2	Survey Methodology	3
2.1	Objective	3
2.2	Survey Design	3
2.3	Distribution	3
2.4	Survey Form Links	3
3	Survey Analysis	4
3.1	Resident (User) Survey Findings	4
3.2	Collector Worker Survey Findings	5
3.3	Cross-Group Analysis	5
4	Selected Features Based on Survey	6
4.1	Resident-Facing Features	6
4.2	Collector Worker Features	7
4.3	Admin Features	7
5	System Design	7
5.1	Architectural Decision: Modular Monolith + ML Microservice	7
5.2	High-Level Architecture Diagram	8
5.3	Architecture Components	9
5.4	Communication Between Monolith and ML Service	9
5.5	Complaint Lifecycle	9
5.6	Data Model Overview	10
5.7	Role-Based Access Control	10
6	ML Service Design	11
6.1	Bengali Complaint Classification	11
6.2	Duplicate Detection	11
6.3	Recurring Hotspot Clustering	11
7	Project Repository Structure	11
8	Deployment Architecture	12
9	Future Scope	13
10	Conclusion	13

1 Introduction

1.1 Background

Dhaka, the capital of Bangladesh, is one of the most densely populated megacities in the world. Every monsoon season, the city faces severe urban waterlogging due to an aging and inadequate drainage infrastructure, unplanned urbanization, and the dumping of solid waste into drainage channels. Waterlogged roads paralyze daily life—disrupting commutes, damaging property, spreading waterborne diseases, and disproportionately affecting low-income communities.

Despite the scale of this recurring crisis, there is **no widely accessible digital platform** that empowers ordinary residents to report drainage problems, track their resolution, or provide municipal drainage workers with the information they need to act efficiently.

1.2 Project Overview

Dubsibhai is a community-driven, AI-enhanced web platform designed to bridge this gap. It allows:

- **Residents** to report waterlogged roads and blocked drains with photos, geolocation, and Bengali text descriptions.
- **Municipal drainage workers (collectors)** to receive complaint assignments, view problem locations on a map, and update resolution status.
- **Administrators** to verify, assign, and analyze complaints at ward and city levels.
- **An ML microservice** to automatically classify Bengali complaints, detect duplicates, and identify recurring waterlogging hotspots using clustering.

Project Goal

To build a collaborative, data-driven drainage management system that transforms how Dhaka handles waterlogging—replacing ad-hoc complaints with structured, prioritized, and traceable reports that reach the right workers at the right time.

1.3 Motivation

The motivation for Dubsibhai stems from three core observations:

1. **Information asymmetry:** Residents experience waterlogging daily but have no structured channel to report it. Workers lack visibility into what problems exist and where.
2. **Lack of prioritization:** Without data, municipal authorities cannot distinguish between a one-time puddle and a chronically waterlogged intersection that needs infrastructure investment.
3. **Opportunity for AI:** Natural language processing on Bengali complaint text, combined with geospatial analysis, can automate categorization, flag duplicate reports,

and surface recurring problem areas—tasks that are tedious and error-prone when done manually.

2 Survey Methodology

2.1 Objective

Before designing the system, we conducted a requirements-gathering survey to understand the actual needs of our two primary stakeholder groups: **residents (end-users)** and **drainage collector workers**. The goal was to validate our feature assumptions and discover unmet needs that should shape the product roadmap.

2.2 Survey Design

Two separate Google Forms were designed, each tailored to its target audience:

Aspect	Resident Survey	Collector Worker Survey
Target audience	General residents of Dhaka who experience waterlogging	Municipal drainage collectors and field workers
Question focus	Desired features, pain points, reporting preferences, expectations from the platform	Information needs, workflow challenges, communication gaps with administration
Question types	Multiple choice, Likert scale, open-ended	Multiple choice, Likert scale, open-ended
Response count	20+ responses	10+ responses

Table 1: Survey design summary

2.3 Distribution

The surveys were distributed via:

- Social media groups and university communities (resident survey)
- Direct outreach to ward-level drainage workers (collector survey)
- Shared through peer networks and class groups

2.4 Survey Form Links

Survey Link

Survey responses (Google Drive):

drive.google.com/drive/folders/11Cs4f_OrDBdZYHXCxVmZsyjMWtMOp1fy

3 Survey Analysis

3.1 Resident (User) Survey Findings

The resident survey revealed strong consensus around several key themes:

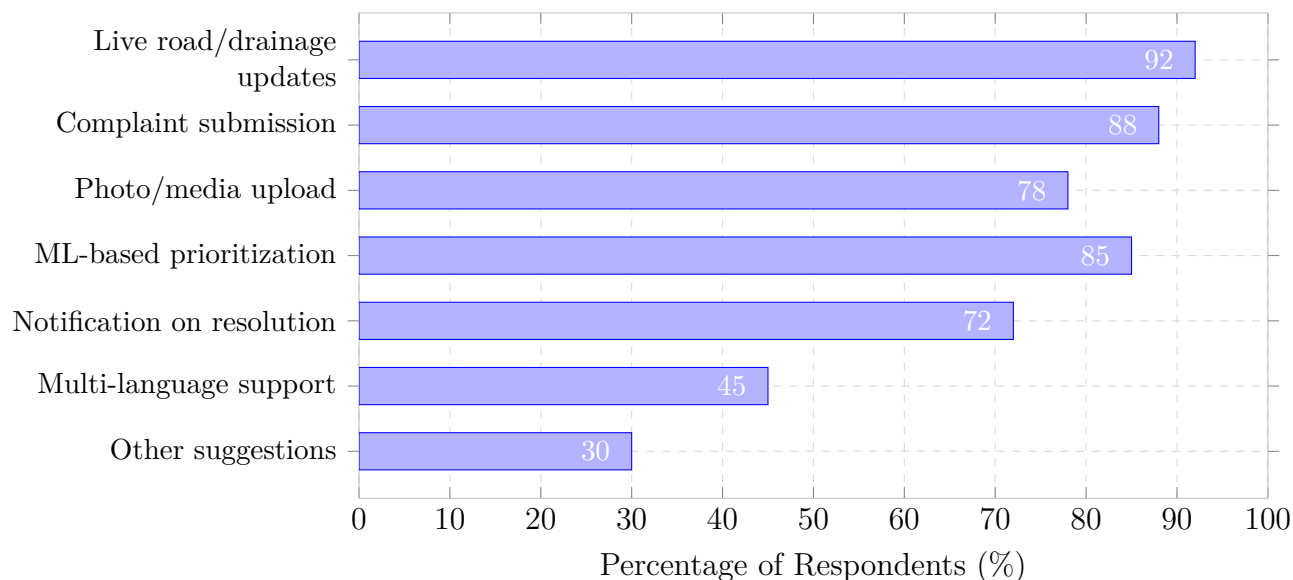


Figure 1: Feature demand from resident survey respondents

Key observations from residents:

1. **Live updates (92%):** The most requested feature. Residents want real-time or near-real-time visibility into which roads are waterlogged, which drains are blocked, and the current status of reported issues.
2. **Complaint submission (88%):** A strong majority want a formal, trackable way to submit drainage complaints—replacing the current informal channels (phone calls, social media posts) that get lost.
3. **ML-based complaint prioritization (85%):** Surprisingly high demand for intelligent prioritization. Residents are frustrated that all complaints are treated equally regardless of severity, and they want the system to automatically escalate critical issues.
4. **Photo/media attachment (78%):** Visual evidence is important for credibility and for helping workers understand the problem before arriving on-site.
5. **Resolution notifications (72%):** Residents want to be notified when their reported issue has been resolved, closing the feedback loop that currently does not exist.

3.2 Collector Worker Survey Findings

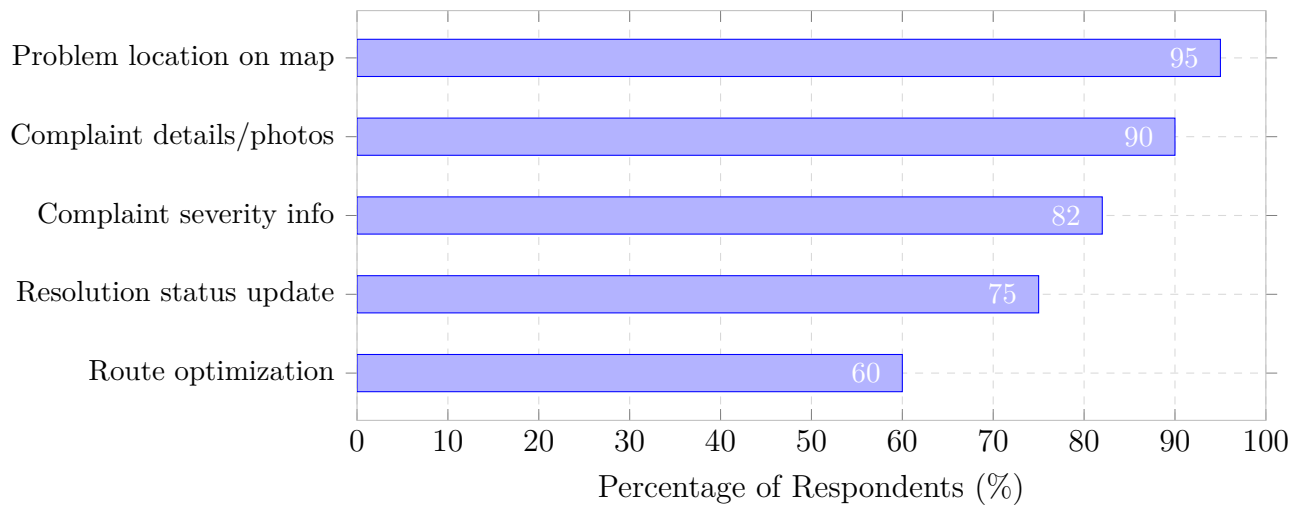


Figure 2: Feature demand from drainage collector worker survey respondents

Key observations from collectors:

1. **Problem location (95%):** The single most critical need. Workers need to know *exactly where* the problem is—a pin on a map, not a vague description like “near the bazar.”
2. **Complaint information (90%):** Workers want access to the full complaint details including photos, textual description, and severity before they head to the site.
3. **Severity classification (82%):** Workers benefit from knowing how urgent a complaint is so they can plan their daily workload effectively.
4. **Status updates (75%):** Workers want to update the resolution status from their end so that both the system and the resident know the problem is being addressed.
5. **Route optimization (60%):** A “nice-to-have” feature where the system could suggest an efficient route to visit multiple complaint sites in a day.

3.3 Cross-Group Analysis

Convergence Between User Groups

Both residents and workers independently identified the same core needs from different perspectives:

- Residents want to *submit* geolocated complaints → Workers want to *receive* geolocated complaints
- Residents want *intelligent prioritization* → Workers want *severity information*
- Residents want *status notifications* → Workers want to *update status*

This convergence validates the platform’s two-sided design and confirms that building features for one group simultaneously benefits the other.

4 Selected Features Based on Survey

Based on the survey results, we finalized the following feature set for DubsiBhai, divided by user role:

4.1 Resident-Facing Features

#	Feature	Description	Priority
1	Live Map Updates	Interactive Leaflet map showing real-time waterlogging status, complaint pins, and drainage condition across Dhaka	High
2	Complaint Submission	Submit complaints with Bengali text description, photo upload, and automatic geolocation capture	High
3	ML Prioritization	Automatic classification of complaint severity and category using a Bengali NLP model (transformer or TF-IDF fallback)	High
4	Status Tracking	Track complaint lifecycle from “Reported” through “Verified,” “Assigned,” “In Progress,” to “Resolved”	Medium
5	Duplicate Detection	ML-powered detection of duplicate complaints about the same issue using text embeddings + geo-proximity	Medium
6	Hotspot Visualization	Heatmap overlay on the map showing recurring waterlogging zones identified by DBSCAN clustering	Medium

Table 2: Selected resident-facing features

4.2 Collector Worker Features

#	Feature	Description	Priority
1	Location View	Map-based view of assigned complaint locations with navigation support	High
2	Complaint Info	Full access to complaint details: text, photos, severity, category, and history	High
3	Status Updates	Ability to update complaint status and add resolution notes with evidence photos	High
4	Assignment Queue	Dashboard showing all current assignments sorted by severity and proximity	Medium

Table 3: Selected collector worker features

4.3 Admin Features

#	Feature	Description	Priority
1	Complaint Verification	Verify or reject incoming complaints before assigning workers	High
2	Worker Assignment	Assign complaints to available workers based on ward and proximity	High
3	Analytics Dashboard	Ward-level and city-wide analytics: resolution rates, average response times, hotspot trends	Medium
4	User Management	Manage resident, worker, and admin accounts; role-based access control	Medium

Table 4: Selected admin features

5 System Design

5.1 Architectural Decision: Modular Monolith + ML Microservice

After evaluating the project requirements, team size, and deployment constraints, we chose a **modular monolithic architecture** for the core application with a **separate ML microservice** for the AI/NLP components.

Why Not Full Microservices?

- **Team size:** A 4-person academic team does not benefit from the operational overhead of distributed microservices.
- **Core app cohesion:** Auth, complaints, assignments, and notifications are naturally cohesive—splitting them adds complexity without benefit.
- **ML justification:** The ML service has a genuinely different runtime (Python ML stack with transformers), different scaling profile (CPU/GPU-bound batch vs. request/response CRUD), and different deploy cadence (model updates shouldn't redeploy the entire app).
- **Fault isolation:** If the ML service is slow or crashes, complaint submission still works—classification runs asynchronously.

5.2 High-Level Architecture Diagram

Figure 3 shows the high-level system architecture of Dubsibhai.

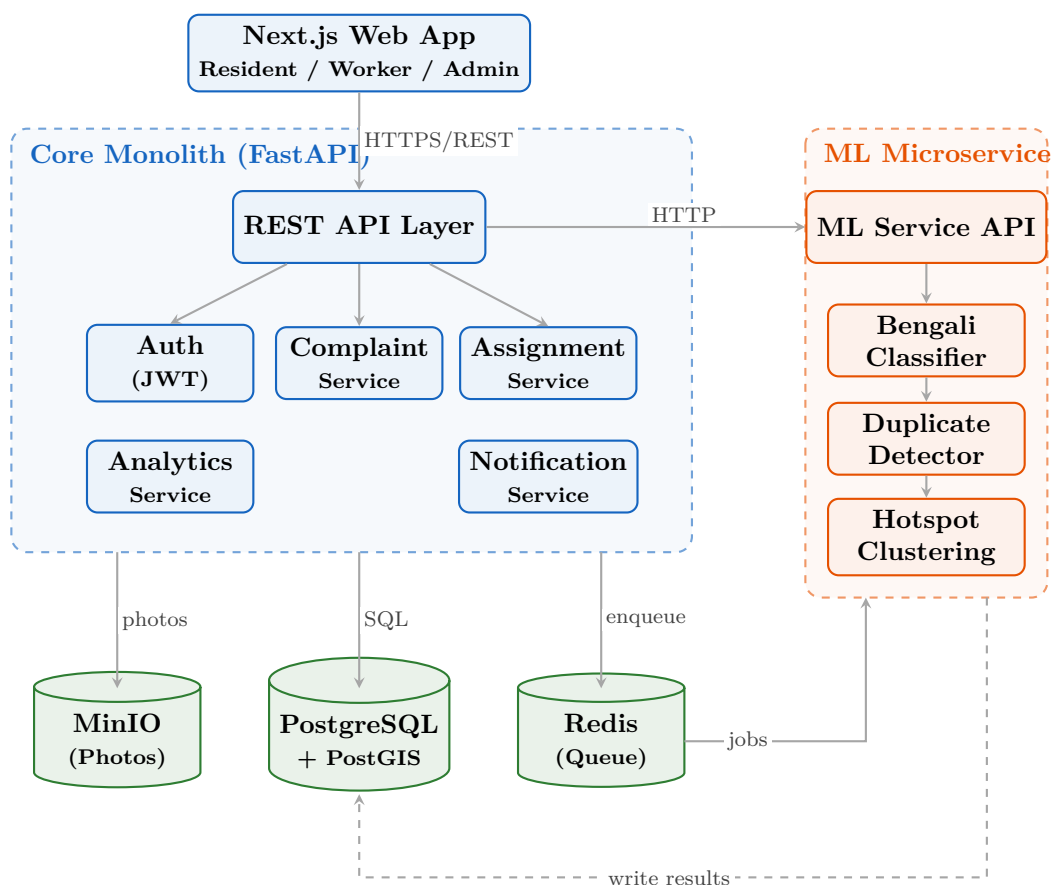


Figure 3: Dubsibhai — High-Level System Architecture

5.3 Architecture Components

Layer	Technology	Purpose
Frontend	Next.js + Tailwind CSS	Responsive web app with Leaflet map integration
Core API	FastAPI (Python)	Modular monolith handling auth, CRUD, assignments, notifications
ML Service	FastAPI + scikit-learn / HuggingFace	Bengali text classification, duplicate detection, clustering
Database	PostgreSQL + PostGIS + pgvector	Relational data, geospatial queries, vector embeddings
Queue	Redis	Async job queue decoupling monolith from ML inference
Storage	MinIO (S3-compatible)	Complaint photo storage
Auth	JWT (python-jose)	Role-based access control (Resident, Worker, Admin)
Maps	Leaflet + OpenStreetMap	Free map tiles for complaint visualization
Deployment	Docker Compose	Six containers: web, api, ml, postgres, redis, minio

Table 5: Technology stack summary

5.4 Communication Between Monolith and ML Service

Async-First Design

Primary (asynchronous): When a resident submits a complaint, the monolith stores it immediately and enqueues a classification job in Redis. The ML service consumes the job, classifies the Bengali text, generates embeddings, checks for duplicates, and writes results back. This keeps complaint submission fast (< 200ms) even if ML inference takes seconds.

Secondary (synchronous): Direct REST calls for on-demand re-classification or admin-triggered clustering batch runs.

5.5 Complaint Lifecycle

The complaint follows a well-defined state machine:

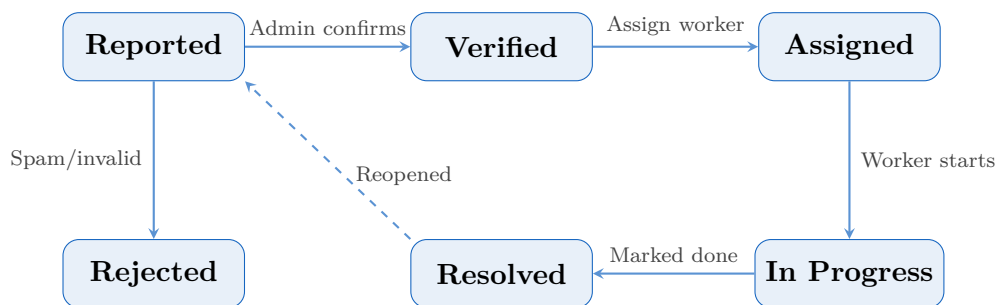


Figure 4: Complaint lifecycle state machine

5.6 Data Model Overview

The core entities and their relationships are:

Entity	Description
User	Residents, workers, and admins with role-based permissions and home area geography
Complaint	Geolocated report with Bengali description, severity, status, and optional duplicate reference
Complaint Photo	S3 keys for uploaded evidence images linked to a complaint
Assignment	Links a worker to a complaint with timestamps and resolution notes
Status Update	Audit trail of every status change with actor and notes
ML Result	Predicted category, confidence, embedding vector, and duplicate candidate IDs
Complaint Cluster	DBSCAN-generated geographic clusters with recurrence classification

Table 6: Core data entities

5.7 Role-Based Access Control

Role	Permissions
Resident	Submit complaints, view own complaints, view public map, comment/upvote
Worker	View assigned complaints, update status, add resolution notes and photos
Ward Admin	Assign workers, verify/reject complaints, view ward-level analytics
Super Admin	Full access, manage users, city-wide analytics, trigger clustering

Table 7: Role-based access control matrix

6 ML Service Design

The ML microservice handles three core AI capabilities:

6.1 Bengali Complaint Classification

- **Input:** Free-text Bengali complaint description (`description_bn`)
- **Output:** Category prediction (blocked drain, open manhole, road flooding, waterlogging-recurring, sewage overflow, other) with confidence score
- **Approach:** TF-IDF + Logistic Regression as the initial baseline, with a path to fine-tune `csebuetnlp/banglabert` for higher accuracy
- **Training data:** Synthetic labeled dataset with planned migration to real complaint data

6.2 Duplicate Detection

- Generate sentence embeddings using a multilingual sentence-transformer
- Store embeddings in `pgvector` (PostgreSQL extension)
- On new complaint: query nearest neighbors within embedding-distance threshold **AND** within a geo-radius (100m via PostGIS `ST_DWithin`) **AND** within a time window (14 days)
- Flagged as candidate duplicate for human review—never auto-merged

6.3 Recurring Hotspot Clustering

- Periodic batch job using DBSCAN clustering on complaint geo-coordinates
- Classifies clusters as *occasional*, *recurring*, or *chronic* based on complaint frequency over time
- Results surfaced as heatmap overlays on the public map

7 Project Repository Structure

DubsiBhai uses a **monorepo** layout with three deployable applications and shared packages:

Monorepo Layout

```
CSE-4113-Project/
|-- apps/
|   |-- web/           # Next.js frontend
|   |-- api/           # FastAPI core monolith
|   '-- ml-service/    # FastAPI ML microservice
|-- packages/
|   |-- shared-types/   # Shared TypeScript types
|   |-- shared-schemas/ # Shared Pydantic schemas
|   '-- ui/             # Shared React components
|-- infra/              # Docker configs, nginx
|-- db/                 # Migrations, seeds
|-- docs/               # Design docs, survey, ADRs
|-- scripts/            # Dev launchers
'-- .github/workflows/  # CI pipelines
```

Rationale:

- **apps/** for deployable services, **packages/** for shared libraries—standard Turborepo/Nx convention
- **db/migrations/** at root level signals “shared resource” since both API and ML service access PostgreSQL
- Per-app CI pipelines (**ci-api.yml**, **ci-ml-service.yml**, **ci-web.yml**) with path-based triggers to avoid unnecessary rebuilds

8 Deployment Architecture

The entire stack is containerized with Docker Compose, consisting of six services:

Service	Port	Description
web	3000	Next.js frontend serving the user interface
api	8000	Core FastAPI monolith handling all business logic
ml-service	8001	ML microservice for classification and analysis
postgres	5432	PostgreSQL with PostGIS and pgvector extensions
redis	6379	Job queue and caching layer
minio	9000	S3-compatible object storage for photos

Table 8: Docker Compose service topology

9 Future Scope

1. **Mobile application:** React Native or Flutter app for easier field reporting by residents and on-the-go status updates by workers
2. **IoT integration:** Connect water-level sensors at known waterlogging points for automated early warning
3. **Route optimization:** AI-powered optimal routing for workers to visit multiple complaint sites efficiently
4. **Predictive analytics:** Use historical data and weather forecasts to predict waterlogging risk before it happens
5. **Multi-city expansion:** Generalize the platform beyond Dhaka to other waterlogging-prone cities in Bangladesh
6. **Public API:** Expose anonymized waterlogging data as an open API for researchers and urban planners

10 Conclusion

This survey report documents the requirements-gathering process for **DubsiBhai**, a community-driven drainage waterlogging management platform for Dhaka. Through targeted surveys of both residents and drainage collector workers, we identified and validated the core features that the platform must deliver:

- **Live waterlogging updates** and an interactive complaint map
- **Structured complaint submission** with photos and geolocation
- **ML-powered prioritization** and duplicate detection using Bengali NLP
- **Worker-facing tools** for location-aware complaint management
- **Administrative oversight** with analytics and assignment workflows

The chosen architecture—a modular monolith for the core application with a separate ML microservice—balances simplicity with the genuine need to isolate the ML runtime. This design is well-justified for a small academic team while remaining extensible for future growth.

Repository

GitHub: <https://github.com/hasanmehediii/CSE-4113-Project>